🎓 Exam-Flow AI Assistant

📤 Upload Assessment

Upload Zoom Recording (MP4)

Drag and drop file here
Limit 200MB per file • …
Browse files

Java_Demo_Exam-Flow.… 42.5MB ✕

🧪 Run Demo Clips

Select one of the pre-provided demo videos to see the AI in action.

🐳 Analyze Docker Demo

☕ Analyze Java Demo

Analysis Complete!

📊 AI Insights

Results for Java Demo

📝 AI Grading Report

Technical Assessment Report

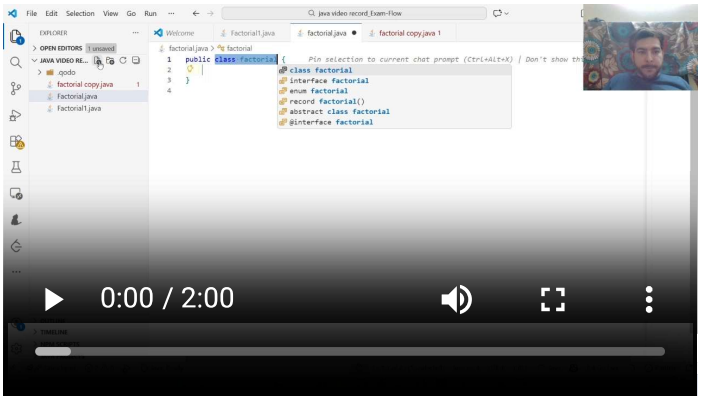
Candidate: Unidentified **Date:** 2/27/2026 10:25 PM

Overview

The candidate was tasked with implementing a Java program to calculate the factorial of a number using recursion. The candidate successfully created a `Factorial` class with a recursive `factorial` method and a `main` method to test it. The program compiled and executed without errors, producing the correct output.

Detailed Analysis & Timestamps

- **00:00 - 00:11:** The candidate starts by defining the `public class Factorial`. Minimal hesitation, showing familiarity with class structure.



Start AI Grading

- **00:11 - 00:14:** Adds a clear comment for "step 1: create a method that calculates factorial of a number using recursion", indicating a planned approach.
- **00:14 - 00:23:** Implements the method signature `public static int factorial(int n)`. This is correctly defined as static and returns an integer.
- **00:23 - 00:28:** Implements the base case for the recursion: `if (n == 0) { return 1; }`. This is essential for preventing infinite recursion and correctly handling factorial of 0.
- **00:28 - 00:33:** Implements the recursive step: `return n * factorial(n - 1);`. This correctly applies the definition of factorial.
- **00:33 - 00:36:** Adds a comment for "step 2: create a main method to test the factorial method". Good practice for code readability.
- **00:36 - 00:41:** Implements the `public static void main(String[] args)` method, which is the entry point for Java applications.
- **00:41 - 00:44:** Declares and initializes an integer variable `int number = 5;` as an example input.
- **00:44 - 00:47:** Calls the `factorial` method with `number` and stores the result in `int result = factorial(number);`. Correct method invocation.
- **00:47 - 00:52:** Uses `System.out.println("Factorial of " + number + " is: " + result);` to print the calculated result to the console.
- **00:52 - 00:58:** Completes the class structure with closing curly braces.
- **00:58 - 01:03:** The file is saved (indicated by the 'unsaved' tag disappearing).

- **01:03 - 01:08:** The candidate initiates the program execution by clicking the 'Run' button.
- **01:08 - 01:15:** The terminal opens and displays the output of the program.
- **01:15 - 01:20:** The console output `Factorial of 5 is: 120` is visible. This is the correct factorial for the input number 5.
- **01:20 - End:** The candidate reviews the successful output.

Grading Rubric Assessment

1. **Correct Java Syntax:**
    - All Java syntax for class declaration, method signatures, conditional statements, variable declarations, and output statements is impeccable.
    - **Verdict:** Excellent.
2. **Implementation of a generic list or loop:**
    - While a generic list was not applicable to this problem, the candidate successfully implemented the factorial logic using **recursion**, which is a valid iterative process for this mathematical function. The recursive definition with its base case and recursive step is perfectly coded.
    - **Verdict:** Excellent.
3. **Successful execution in console:**
    - The program compiled without errors and executed successfully. The output `Factorial of 5 is: 120` is accurate, demonstrating that the logic functions as intended.
    - **Verdict:** Excellent.

Behavioral Red Flags

- No behavioral red flags were observed. The candidate remained focused on the screen, typed consistently, and showed no signs of distraction or external assistance. Eye movements were consistent with someone actively coding and reviewing their work.

Final Score

100/100

Justification

The candidate demonstrated an exceptional understanding of Java programming concepts. The implementation of the factorial function using recursion was not only syntactically perfect but also logically sound, correctly handling both the base case and the recursive step. The program executed flawlessly, yielding the correct result in the console. The inclusion of comments for each step indicates a clear thought process and good coding practices. The candidate completed the task confidently and efficiently without any discernible errors or external assistance.

🚨 Behavioral Flags (MediaPipe)

00:00: Proctoring module disabled (Missing Dependencies) (Info)